Design a Dynamic Web App — Fashion Store / E-Commerce Product Page Using HTML, CSS & JavaScript

1. Objectives

- To design and develop a fully dynamic, interactive fashion e-commerce web application using HTML5, CSS3, and vanilla JavaScript — without any external frameworks or libraries.
- To implement core JavaScript concepts such as DOM manipulation, event handling, array methods (filter, sort, map), and localStorage to create a functional product listing, search, cart, and wishlist system.
- To build a responsive, visually appealing front-end interface that simulates a real-world online fashion store with features like category filtering, live search, product modals, a shopping cart drawer, and toast notifications.

2. Software / Hardware Requirements

Software Requirements:

- Operating System: Windows 10/11, macOS, or any Linux distribution
- Code Editor: Visual Studio Code, Sublime Text, or Notepad++
- Web Browser: Google Chrome, Mozilla Firefox, or Microsoft Edge (latest version)
- Languages: HTML5, CSS3, JavaScript (ES6+) — no frameworks or external libraries
- Google Fonts: Loaded via CDN for custom typography (Cormorant Garamond, Jost)
- Server: Not required — file runs directly in browser as a local HTML file

Hardware Requirements:

- Processor: Any modern CPU (Intel Core i3 or equivalent)
- RAM: Minimum 2 GB (4 GB recommended)
- Storage: Minimum 10 MB free space

- Display: Resolution of 1024x768 or higher

3. Theory

3.1 Introduction to Dynamic Web Applications

A dynamic web application differs from a static web page in that it responds to user interactions and updates the content on screen without requiring a full page reload. This is achieved through JavaScript, which manipulates the Document Object Model (DOM) in real time based on user events such as clicks, typing, and scrolling.

An E-Commerce or fashion store web app is an excellent real-world project to apply these concepts. It involves displaying product data, allowing users to filter and search, view product details, manage a shopping cart, and persist data across sessions — all requiring a combination of HTML structure, CSS styling, and JavaScript logic.

3.2 Role of HTML, CSS, and JavaScript

- HTML5: Provides the semantic structure — navbar, hero, product grid, modal, cart drawer, footer. Uses tags like <nav>, <section>, , <button>, <input>, <select>, <img>.
- CSS3: Handles all visual styling — dark luxury theme, grid layouts, hover animations, modal overlays, slide-in cart, responsive breakpoints via media queries, and keyframe animations for toast notifications.
- JavaScript (ES6+): Powers all dynamic behaviour — rendering product cards from a data array, filtering/sorting, opening modals, managing cart state, updating totals, and saving data to localStorage.

3.3 Application Features

The following table lists all features implemented in the dynamic fashion store web app:

Feature	Description	Technology Used
Hero Banner	Full-screen landing section with headline, tagline, CTA button	CSS animation, flexbox, overlay

Feature	Description	Technology Used
Product Grid	Dynamic product cards rendered via JS — image, name, price, rating	JS DOM, CSS Grid, template literals
Category Filter	Filter buttons to show products by Men / Women / Accessories / All	Array.filter(), DOM classList toggle
Live Search	Real-time search as user types — filters by product name instantly	input event, Array.filter(), regex
Sort Products	Sort by: Price Low-High, Price High-Low, Rating, Newest	Array.sort(), select change event
Product Modal	Click any product to open detail popup with size selector and Add to Cart	JS modal, dynamic innerHTML, overlay
Shopping Cart	Slide-in cart drawer showing items, quantities, and total price	JS array state, localStorage, flex
Add / Remove	Add items, increase/decrease quantity, remove from cart	Array splice, find, map, re-render
Cart Badge	Live cart item count displayed on the cart icon in the navbar	DOM update on state change
Toast Notification	Brief popup message when item is added to cart	CSS keyframe animation, setTimeout
Responsive Layout	Mobile-friendly grid that collapses on small screens	@media queries, CSS Grid auto-fit
Wishlist Toggle	Heart icon on each card to save/remove favourite products	JS Set, DOM toggle, localStorage

3.4 Key JavaScript Concepts Used

The following table summarises the key JavaScript concepts applied to build the dynamic functionality of the app:

JS Concept	How It Is Used in the App
Array.filter()	Filter products by category and search query
Array.sort()	Sort product list by price or rating
localStorage	Persist cart and wishlist data between page refreshes
Event Listeners	Handle clicks, input changes, modal open/close, cart toggle
Template Literals	Build dynamic HTML strings for product cards and cart items
DOM Manipulation	Render, update, and clear product grids and cart drawer dynamically

JS Concept	How It Is Used in the App
Spread Operator	Copy arrays before sorting to avoid mutating original data
Modules / State	Maintain a central products array and cart array as app state

3.5 Application Architecture

The app follows a simple single-file architecture — all HTML, CSS, and JavaScript are contained within one .html file, making it easy to open and run in any browser without a server. The JavaScript section maintains two key state arrays: a products array (containing all product objects with id, name, category, price, rating, and image) and a cart array (containing items the user has added). All UI updates are driven by these state arrays — whenever the state changes, the relevant portion of the DOM is re-rendered.

The product images are loaded from a free placeholder/image service (Unsplash/Picsum) to simulate real product photos. The cart and wishlist are saved to the browser's localStorage so data is preserved when the user refreshes the page.

4. Conclusion / Analysis

This assignment successfully demonstrated the development of a fully functional dynamic e-commerce web application using only HTML5, CSS3, and vanilla JavaScript. The project brought together a wide range of front-end concepts in a practical, real-world context.

A key learning was understanding how JavaScript bridges HTML structure and CSS styling to create interactive experiences. The DOM manipulation — dynamically rendering product cards, opening modals, updating cart counts, and showing toast notifications — illustrated how modern web apps work without requiring page reloads.

Working with JavaScript array methods such as filter(), sort(), and map() reinforced their importance in data-driven UI development. These methods are fundamental to how real e-commerce platforms like Flipkart, Amazon, and Myntra handle product listing, filtering, and sorting on the client side.

The use of localStorage for persisting cart and wishlist data introduced the concept of client-side storage — a core technique used in real applications before a backend or database is introduced. Overall, this

project provided a strong practical foundation in dynamic front-end web development, demonstrating that a fully interactive and responsive e-commerce UI can be built with just three core web technologies.